

Bike Rental Demand Forecasting

UE24CS352A · Machine Learning Mini-Project · Batch 19

PES2UG24CS063 · Aniket Raosaheb Jadhav | PES2UG24CS109 · Basava Prabhu

1 What we had to do

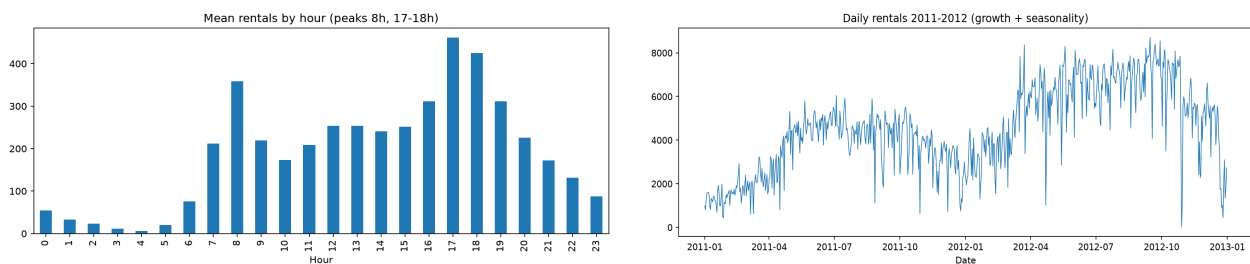
Bike-sharing apps need to know how many bikes will be needed each hour so they can place enough bikes at the right stations. We used two years of Washington D.C. rental records (2011–2012) along with the time and weather for each hour, and built a system that predicts the total number of rentals.

We split the data by time — train on the past, test on the most recent weeks — so the test feels like the real future. We compared four approaches and kept the one that was most reliable.

2 The data

Two files from the same source: **hour.csv** with 17,379 hourly records and **day.csv** with 731 daily records. Each row has the date, season, month, weekday, holiday, working-day flag, weather type, temperature, feels-like temperature, humidity and wind speed. The answer we predict is **count**, the total rentals (casual + registered users). The data is clean: no missing values and no duplicates.

Two things stood out early. Rentals grow year over year and follow seasons — autumn is busiest, spring is quietest. And the hour of the day matters more than anything else: mornings around 8 and evenings around 5–6 are very busy on working days.



Left: average rentals by hour — clear morning and evening peaks. Right: daily rentals over two years — growth plus seasonal waves.

3 How we built it

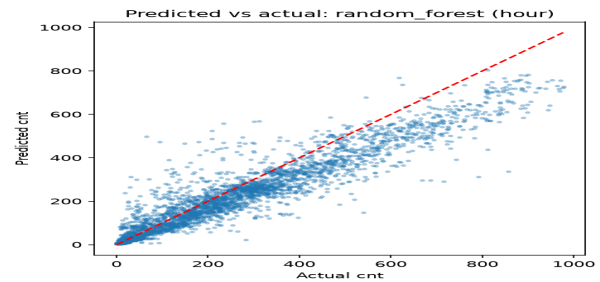
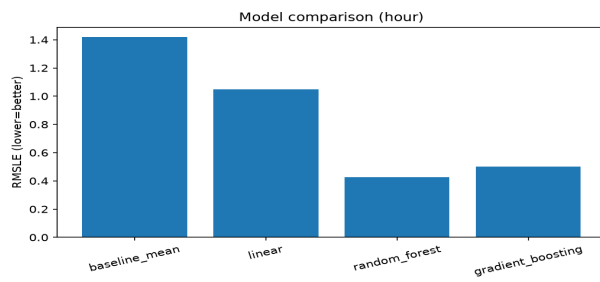
First we turned the date into useful parts: hour, weekday, month, a weekend flag, and rush-hour flags for weekday and weekend peaks. Because 11pm and midnight are next to each other, we also added circular encodings so the model sees that. Temperature and feels-like temperature move together almost exactly, so we kept only the feels-like one. Season and weather type were treated as categories, not numbers.

All scaling and encoding was fitted on past data only and then applied to future data, so no future information leaks into training. We then trained four models: a simple average baseline, linear regression, random forest, and gradient boosting.

4 What we found

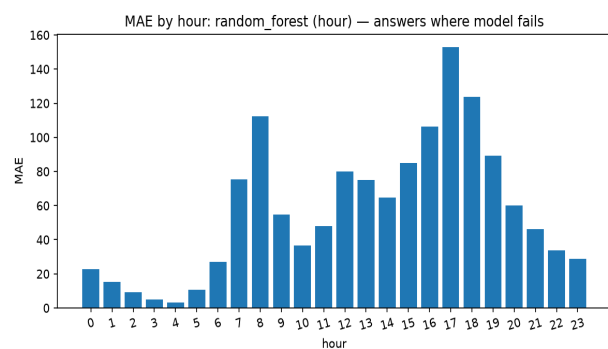
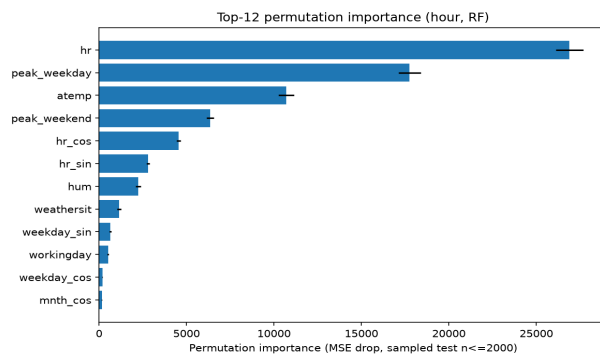
Model (hourly test)	Avg. error	Large-error penalty	% style error
Gradient boosting	52.6 bikes	77.3	0.50
Random forest	57.1 bikes	84.8	0.43 (best)
Linear regression	94.3 bikes	126.7	1.05
Average baseline	176.8 bikes	237.7	1.42

On daily data the gap is similar: the best model misses by about 880–1,070 bikes a day against a daily average of ~4,500, while guessing the overall average misses by ~2,550. Tree-based models win because rental patterns are not straight lines — rush hours, weekends and weather interact.



Left: lower is better — trees clearly beat the baseline. Right: predicted vs actual rentals clustering near the diagonal.

The hour of the day is by far the most useful signal, followed by the rush-hour flags and feels-like temperature. Removing any group — rush-hour flags, time cycles, or weather — makes results worse. Errors are highest during the 5pm peak and in bad weather, and high-demand hours naturally have larger absolute errors.



Left: what the model relies on most. Right: where it still struggles — the evening peak.

5 How to run it & who did what

Code lives in **src/** (cleaning, training, evaluation, prediction), analysis in **notebooks/01_eda.py**, figures in **reports/figures**. Aniket handled data cleaning and exploration; Basava handled training and validation; evaluation, report and demo were joint. To reproduce: **pip install -r requirements.txt**, then run the EDA, training, and evaluation scripts in order — see README. A demo script predicts rentals for any hour, e.g. 8am on a working day.

6 Takeaway

A small, honest setup works: good time features plus a tree model predicts hourly demand far better than averages or straight-line models. The main limits are rare bad-weather hours (very few examples) and the small daily dataset. Next we would tune the models and study peak-hour errors more closely.